

Documentation Technique du Backend

Systeme de Gestion des Absences - Maroc YNOV Campus

Stage de 2 mois (Aout - Septembre 2026)

Date de generation : 24 Aout 2026

1. Introduction et Contexte

Ce document presente l'etat actuel du backend de l'application de Gestion des Absences developpe dans le cadre du stage au sein de Maroc YNOV Campus. Le backend est construit avec les technologies suivantes :

- **Node.js** et **Express.js** pour le serveur API RESTful
- **Firebase Authentication** pour l'authentification securisee des utilisateurs
- **Firebase Firestore** pour le stockage des donnees (base de donnees NoSQL)
- **Nodemailer** pour l'envoi d'e-mails HTML personnalises (alertes RH, decisions)
- **Multer** pour la reception des fichiers telecharges (justificatifs)
- **Stockage Local sur Disque** (`/uploads/`) pour l'hebergement des fichiers justificatifs

Controle d'accès : Le systeme utilise le mecanisme RBAC (Role-Based Access Control) via les Firebase Custom Claims pour gerer 7 roles systeme : **Admin, RH, Manager, Teacher, Employee, Student** et **Parent**.

2. Architecture du Projet

Le backend suit une architecture en 3 couches (Validator → Service → Controller) pour chaque module fonctionnel :

```
backend/
├── Absence/                                # Module Gestion des Absences
│   ├── Controllers/
│   │   └── absenceController.js           # 118 lignes - 7 handlers Express
│   ├── Services/
│   │   └── absenceService.js              # 263 lignes - Logique metier Firestore + Emails + Notifications
│   └── Validators/
│       └── absenceValidator.js            # 64 lignes - Validation des champs et statuts
├── Auth/                                   # Module Authentification et Utilisateurs
│   ├── Authentication/
│   │   ├── authController.js              # Connexion, deconnexion, reinitialisation
│   │   ├── authService.js                 # Services Firebase Auth
│   │   └── customEmailService.js          # Envoi d'emails HTML (alerte RH, decision absence)
│   └── Users/
│       ├── userController.js              # Creation comptes, liaison Parent-Etudiant
│       └── userService.js                 # Services Admin SDK Firestore
└── Roles & Permissions/
```

```

|       └─ roleController.js           # Assignment de roles
|       └─ roleService.js             # Gestion Custom Claims Firebase
|
└─ Documents/                         # Module Televersement de Justificatifs
    └─ Controllers/
        └─ documentController.js      # Reception fichier via Multer
    └─ Services/
        └─ documentService.js         # Stockage disque local + metadonnees Firestore
    └─ Validators/
        └─ documentValidator.js       # Validation taille (5 Mo max) et types MIME
|
└─ Notifications/                    # Module Notifications In-App
    └─ Controllers/
        └─ notificationController.js  # Consultation et marquage comme lu
    └─ Services/
        └─ notificationService.js     # Creation, lecture, mise a jour notifications
|
└─ Shared/                           # Composants partages
    └─ Authentication middleware/
        └─ authMiddleware.js          # Middleware Token Bearer + Verification RBAC
    └─ Roles/
        └─ roles.js                  # Constantes des 7 roles systeme
    └─ Permissions/
        └─ permissions.js            # Constantes des permissions
    └─ Utilities/
        └─ utilities.js               # Formateurs de reponses API
|
└─ uploads/                          # Dossier local des fichiers telecharges
└─ server.js                          # Point d'entree du serveur Express (94 lignes)
└─ firebaseAdmin.js                   # Configuration Firebase Admin SDK
└─ firebaseConfig.js                  # Configuration Firebase Web Client SDK
└─ seed.js                            # Script d'initialisation des 7 comptes par default
└─ YNOV_Absence_API.postman_collection.json # Collection de tests Postman

```

3. Modules Fonctionnels Implementes

Statut global : Tous les modules decrits ci-dessous sont **entierement implementes**, avec leurs routes enregistrees dans `server.js`, leurs validateurs, services metier, et controleurs Express.

3.1. Module Authentication (`/api/auth`)

Gere la connexion, la deconnexion et la reinitialisation du mot de passe via Firebase Authentication et Nodemailer.

METHODE	ENDPOINT	ACCES	DESCRIPTION
POST	<code>/api/auth/login</code>	Public	Authentifie un utilisateur avec email et mot de passe. Renvoie le jeton ID Token Firebase et les informations de profil (role, departement).
POST	<code>/api/auth/reset-password</code>	Public	Genere un lien securise de reinitialisation de mot de passe et l'envoi par e-mail HTML personnalise via Nodemailer SMTP.
POST	<code>/api/auth/logout</code>	Public	Deconnecte l'utilisateur et invalide sa session.

3.2. Module Gestion des Utilisateurs et Roles (`/api/users` et `/api/roles`)

Gere la creation de comptes par les administrateurs, l'assignation de roles et la liaison Parent-Etudiant.

METHODE	ENDPOINT	ACCES	DESCRIPTION
GET	<code>/api/users</code>	Authentifie	Recupere la liste de tous les comptes utilisateurs depuis Firestore.
POST	<code>/api/users/create</code>	Admin / RH	Cree un compte utilisateur dans Firebase Auth avec Custom Claims et enregistre son profil dans Firestore.
POST	<code>/api/roles/assign</code>	Admin	Assigne ou modifie le role Custom Claim d'un utilisateur existant.
POST	<code>/api/users/link-parent-student</code>	Admin / RH	Lie un compte Parent a un compte Etudiant dans Firestore (<code>childrenUids</code> et <code>parentUids</code>).
GET	<code>/api/users/my-children</code>	Parent / Admin / RH	Recupere les profils des etudiants lies au compte Parent connecte.

3.3. Module Gestion des Absences (`/api/absences`)

Ce module est le coeur fonctionnel de l'application. Il permet la soumission, la consultation, la revision (approbation/rejet) et la suppression des demandes d'absence. Chaque action est securisee par le middleware d'authentification Bearer Token et le controle RBAC.

Statut : Entierement implemente. Le module comprend 3 fichiers complets : `absenceController.js` (118 lignes, 7 handlers), `absenceService.js` (263 lignes, logique metier complete), `absenceValidator.js` (64 lignes, validation des champs). Les 7 routes sont declarees dans `server.js` aux lignes 68-75.

METHODE	ENDPOINT	ACCES	DESCRIPTION
POST	<code>/api/absences</code>	Authentifie	Soumet une nouvelle demande d'absence. Declenche automatiquement l'alerte e-mail RH et la notification In-App a l'utilisateur.
GET	<code>/api/absences/my</code>	Authentifie	Recupere l'historique des absences soumisees par l'utilisateur connecte.
GET	<code>/api/absences/children</code>	Parent / Admin / RH	Permet aux parents de consulter les demandes d'absence de leurs enfants lies via <code>childrenUids</code> .
GET	<code>/api/absences/pending</code>	Admin / RH / Manager	Recupere toutes les demandes d'absence en attente de revision (statut <code>pending</code>).
GET	<code>/api/absences</code>	Admin / RH / Manager / Teacher	Recupere l'ensemble des absences avec possibilite de filtrer par <code>status</code> , <code>department</code> ou <code>type</code> .
PATCH	<code>/api/absences/:id/review</code>	Admin / RH / Manager	Approuve ou rejette une demande d'absence. Declenche l'e-mail de decision et la notification In-App a l'etudiant.
DELETE	<code>/api/absences/:id</code>	Proprietaire uniquement	Annule et supprime une demande d'absence uniquement si elle est encore en statut <code>pending</code> et appartient a l'utilisateur.

Types d'absence autorises

VALEUR	DESCRIPTION
<code>medical</code>	Absence pour raison medicale
<code>personal</code>	Absence pour motif personnel ou familial
<code>authorized_leave</code>	Conge autorise
<code>unjustified</code>	Absence non justifiee
<code>other</code>	Autre type d'absence

Statuts de la demande d'absence

STATUT	DESCRIPTION
<code>pending</code>	En attente de revision par l'administration
<code>approved</code>	Demande approuvee par un Admin / RH / Manager
<code>rejected</code>	Demande rejetee par un Admin / RH / Manager

Validations effectuees lors de la soumission

- Le champ `reason` (motif) est obligatoire et doit etre une chaine non vide.
- Le champ `type` doit correspondre a l'une des valeurs autorisees (medical, personal, authorized_leave, unjustified, other).
- Les champs `startDate` et `endDate` sont obligatoires et doivent etre des dates valides.
- La date de debut ne peut pas etre posterieure a la date de fin.

Automatisations declenchees

- **Lors de la soumission** (`POST /api/absences`) : envoi automatique d'un e-mail d'alerte au service RH via Nodemailer + creation d'une notification In-App pour l'utilisateur.
- **Lors de la revision** (`PATCH /api/absences/:id/review`) : envoi automatique d'un e-mail de decision (approbation ou rejet) a l'etudiant/employe + creation d'une notification In-App.

3.4. Module Televersement de Justificatifs (`/api/documents/upload`)

Permet aux utilisateurs de televerser des fichiers justificatifs (certificats medicaux, attestations) qui sont stockes localement sur le disque du serveur et dont les metadonnees sont enregistrees dans la collection Firestore `documents` .

METHODE	ENDPOINT	ACCES	DESCRIPTION
POST	<code>/api/documents/upload</code>	Authentifie	Receptionne un fichier via Multer (form-data, cle <code>file</code>), le sauvegarde dans <code>backend/uploads/justifications/{userId}/</code> , et cree le document de metadonnees dans Firestore.

Contraintes de validation

- Taille maximale du fichier : **5 Mo**
- Types MIME autorises : `application/pdf` , `image/png` , `image/jpeg`

Schema des metadonnees Firestore (collection `documents`)

CHAMP	TYPE	DESCRIPTION
<code>id</code>	String	Identifiant unique du document Firestore
<code>uid</code>	String	UID de l'utilisateur propriétaire
<code>originalName</code>	String	Nom original du fichier telecharge
<code>filename</code>	String	Nom du fichier sur le disque local
<code>mimeType</code>	String	Type MIME du fichier
<code>size</code>	Number	Taille du fichier en octets
<code>category</code>	String	Categorie (justificatif, autre)
<code>status</code>	String	Statut du document (validated)
<code>storageArea</code>	String	Zone de stockage (justificatifs)
<code>storagePath</code>	String	Chemin relatif du fichier sur le serveur
<code>url</code>	String	URL HTTP publique pour telecharger le fichier
<code>archived</code>	Boolean	Indique si le document est archive
<code>createdAt</code>	Timestamp	Date de creation
<code>updatedAt</code>	Timestamp	Date de derniere mise a jour

3.5. Module Notifications In-App (`/api/notifications`)

Gere les notifications internes de l'application stockees dans la collection Firestore `notifications` . Les notifications sont creees automatiquement lors de la soumission et de la revision des absences.

METHODE	ENDPOINT	ACCES	DESCRIPTION
GET	<code>/api/notifications/my</code>	Authentifie	Recupere la liste des notifications de l'utilisateur connecte avec le compteur de notifications non lues.
PATCH	<code>/api/notifications/:id/read</code>	Authentifie	Marque une notification specifique comme lue (<code>read: true</code>).
POST	<code>/api/notifications/read-all</code>	Authentifie	Marque toutes les notifications de l'utilisateur comme lues en une seule operation.

4. Securite et Controle d'Acces (RBAC)

4.1. Middleware d'Authentification

Chaque requete protegee passe par le middleware `authenticateToken` qui :

1. Verifie la presence du header `Authorization: Bearer <ID_TOKEN>`
2. Valide le jeton Firebase ID Token via le SDK Firebase Admin
3. Decode les Custom Claims (role, permissions) et les attache a `req.user`

4.2. Middleware de Verification des Roles

Le middleware `authorizeRoles(...roles)` verifie que le role de l'utilisateur authentifie est inclus dans la liste des roles autorises pour l'endpoint.

4.3. Matrice des Roles et Acces

ROLE	CONSTANTE	DROITS PRINCIPAUX
Admin	<code>ROLES.ADMIN</code>	Acces complet a toutes les fonctionnalites. Creation de comptes, assignation de roles, revision des absences, liaison Parent-Etudiant.
RH	<code>ROLES.RH</code>	Creation de comptes, revision des absences, consultation de toutes les absences, liaison Parent-Etudiant.
Manager	<code>ROLES.MANAGER</code>	Revision des absences, consultation des demandes en attente et de l'ensemble des absences.
Teacher	<code>ROLES.TEACHER</code>	Consultation de l'ensemble des absences (lecture seule).
Employee	<code>ROLES.EMPLOYEE</code>	Soumission de demandes d'absence, consultation de son historique, televersement de justificatifs.
Student	<code>ROLES.STUDENT</code>	Soumission de demandes d'absence, consultation de son historique, televersement de justificatifs.
Parent	<code>ROLES.PARENT</code>	Consultation des absences de ses enfants lies, consultation des profils enfants.

5. Comptes de Test par Defaut (Seed)

Le script `seed.js` cree automatiquement 7 comptes de demonstration dans Firebase Authentication et Firestore :

ROLE	EMAIL	MOT DE PASSE
Admin	amine.fatih@ynov.com	AdminPassword123!
RH	rh@ynov.com	RhPassword123!
Manager	manager@ynov.com	ManagerPassword123!
Teacher	enseignant@ynov.com	TeacherPassword123!
Employee	employee@ynov.com	EmployeePassword123!
Student	etudiant@ynov.com	StudentPassword123!
Parent	parent@ynov.com	ParentPassword123!

```
cd backend
node seed.js
```

6. Configuration et Variables d'Environnement

Le fichier `.env` du backend contient les variables suivantes :

VARIABLE	DESCRIPTION	EXEMPLE
PORT	Port d'ecoute du serveur Express	5000
SMTP_HOST	Serveur SMTP pour l'envoi d'emails	smtp.gmail.com
SMTP_PORT	Port SMTP	587
SMTP_USER	Adresse email SMTP	crafterscode01@gmail.com
SMTP_PASS	Mot de passe d'application SMTP	votre_mot_de_passe_app
HR_EMAIL	Adresse email de destination des alertes RH	rh@ynov.com

Fichier Firebase Service Account : Le fichier JSON de compte de service Firebase Admin SDK (`backend-91067-firebase-adminsdk-fbsvc-*.json`) doit etre place dans le dossier racine du projet (au meme niveau que le dossier `backend/`).

7. Guide de Demarrage

Etape 1 : Installation des dependances

```
cd backend
npm install
```

Etape 2 : Configuration de l'environnement

- Copier `.env.example` vers `.env` et renseigner les identifiants SMTP.
- Placer le fichier JSON Firebase Service Account dans le dossier racine du projet.

Etape 3 : Initialisation des comptes de test

```
node seed.js
```

Etape 4 : Demarrage du serveur

```
# Mode standard
npm start

# Mode developpement (rechargement automatique)
npm run dev
```

Le serveur démarre sur `http://localhost:5000` .

Etape 5 : Verification

```
GET http://localhost:5000/api/health
```

Reponse attendue :

```
{ "success": true, "message": "API Backend Gestion des Absences opérationnelle." }
```

8. Resume des Endpoints (19 Routes)

#	METHODE	ENDPOINT	MODULE	ACCES
1	POST	/api/auth/login	Auth	Public
2	POST	/api/auth/reset-password	Auth	Public
3	POST	/api/auth/logout	Auth	Public
4	GET	/api/users	Users	Authentifie
5	POST	/api/users/create	Users	Admin / RH
6	POST	/api/roles/assign	Roles	Admin
7	POST	/api/users/link-parent-student	Users	Admin / RH
8	GET	/api/users/my-children	Users	Parent / Admin / RH
9	POST	/api/absences	Absences	Authentifie
10	GET	/api/absences/my	Absences	Authentifie
11	GET	/api/absences/children	Absences	Parent / Admin / RH
12	GET	/api/absences/pending	Absences	Admin / RH / Manager
13	GET	/api/absences	Absences	Admin / RH / Manager / Teacher
14	PATCH	/api/absences/:id/review	Absences	Admin / RH / Manager
15	DELETE	/api/absences/:id	Absences	Proprietaire
16	POST	/api/documents/upload	Documents	Authentifie
17	GET	/api/notifications/my	Notifications	Authentifie
18	PATCH	/api/notifications/:id/read	Notifications	Authentifie
19	POST	/api/notifications/read-all	Notifications	Authentifie